Backlog dependency graph

This is the canonical ordering for the remaining work. It exists because the same issue often appears under several names. Solve the roots first; do not start a leaf that will be rewritten when its blocker lands.

Ground rule

Every item follows the same workflow: documentation first, tests derived from the documentation, implementation to green, then the specification and counts updated in the same work item. A change that adds execution state also adds a root-registry entry and a persistence test. A change that adds a product mode also adds an AOT smoke.

Agreement. Yes to ground-up, and the complete plan is adopted. No feature is deferred for code size. R0 is the shared callable and recursive-group substrate. R1 is the full recursion surface and F# interop: `logic` and `cycle` definitions, F#-style `and`, our own F# abstraction, the build-time classifier, the manifest and tiers, diagnostics, image stages, and IPC metadata. R2 is the full book model: recursive groups, with `cycle` marking recursive definitions and `and` grouping them. D194 lands the group and markers; D189 nets, not the target.

The graph

```
R0  callable/call semantics + recursive group substrate  (root)
     |  resolve a name to a callable; distinguish reference from call
     |  UnitGroup/Unit, arity, captures, InitOrder, depth guard
     |  named refusal until the group runtime lands
     +--> R1  complete recursion surface + our own F# abstraction
     |         logic/cycle + F#-style and + F# read-off + manifest/tiers
     |         diagnostics + image stages + IPC metadata + versioning
     |         |
     |         +--> R2  book recursion: groups + logic/cycle markers
     |                  |
     |                  +--> R3  book evaluation/package contract
     |                           (lazy/eager, main arity, cache, depth, imports)
     +--> R4  effect port / capability manifest
     |         |
     |         +--> R10 Stage 2 shell (streams, pipelines)
     |         +--> R12 security sandbox (allowed programs/paths, env)
     +--> R6  typecheck/signatures (host words, books, retrieval vs checking)
     +--> R8  AOT smoke for --prose/--book/--cast

R5  root registry / persistence   (state foundation; before new state)
R7  benchmark baseline            (independent; before performance work)
R9  docs split / generated status (independent; reduces reading cost)
R11 code renames / verification move (after R1--R6 settle)
R13 semantic test matrix          (parallel; each item owns its slice)
```

Id	Item	Blocked by / why
R0	Callable/call semantics and the recursive group substrate: one resolution; a call is not a reference; `UnitGroup`/`Unit` with arity, captures, `InitOrder`, and a depth guard; named refusal until the group runtime lands	The root. `CallWord` currently serves both a word call and a name reference; the group substrate is the one way a self or mutual call becomes a real call. This blocks R1, R2, R6, and the host-word/word-table work.
R1	Complete recursion surface and our own F# abstraction: `logic`/`cycle`, F#-style `and`, F#	R0 for the substrate; R5 for staged roots; design records for manifest, diagnostic

Id	Item	Blocked by / why
	read-off, build-time classifier, manifest and tiers, diagnostics, image stages, IPC metadata, versioning	mapping, image stages, the F# abstraction subset, and versioning. See `docs/recursion.typ`.
R2	Full book recursion: automatic recursive groups by default, plus optional explicit markers checked against the computed group	R0/R1. The book is a module group; the explicit markers are an optional refinement, not a second recursion model. D194 lands the automatic group.
R3	Book evaluation and package contract: lazy/eager, `main` arity and side effects, package imports/capabilities/versioning, unit hashes/dependencies, cache, depth/cycle policy, path policy	R2 for the recursion/evaluation order; R4 for capability declarations; R5 for any new state.
R4	Effect port and capability manifest: one seam for `scale_host_call` and in-process host words; arity validation; default-deny; image/package declares required effects	R0 for host-word resolution; the capability design record. Independent of R1/R2 but blocks R10/R12 and the AOT effect path.
R5	Done: root registry (`RootKind`/`rootKinds`, `getRoot`/`setRoot`), set/get round-trip, store round-trip, and a reflection coverage test for new roots	Landed on 2026-11-04. It must stay ahead of R2/R3/R4 state, otherwise new state will be added without a root.
R6	Typecheck/signatures: host-word signatures; book typechecking; distinguish `TypeRetrieval` from checking; a `--check` gate	R0 for callable signatures; R1/R2/R3 for the book and recursion shapes.
R7	Done: benchmark baseline (`--benchmark`, `docs/performance-baseline.typ`); representative programs, wall time, allocations; the AOT smoke is R8	Landed on 2026-11-04; the first JIT reference is committed. It stays evidence, not a rewrite.
R8	AOT smoke for `--prose`, `--book`, `--cast`; product modes in the release gate	R3/R4 for the modes and effects; a product AOT build.
R9	Done: docs split; `docs/roadmap.typ` current, `docs/roadmap-archive.typ` history, generated `docs/decision-index.typ`, generated status/line counts, and a read budget	Landed on 2026-11-04; it reduces the reading cost of every other item.
R10	Stage 2 shell: streams, pipelines, file redirection	R4; the shell effect model must be settled first. Not blocked by recursion.
R11	Code renames (`SymbolicAbstraction`, `ExpressionLogic`, `TypeRetrieval`, etc.) and the unified `verification/` directory	R1–R6 settle; a rename during a semantic change creates a review that cannot be read.
R12	Capability sandbox: allowed programs/paths, cleared environment, book capabilities, package signatures	R4 and R3; the effect manifest is the permission surface.

Id	Item	Blocked by / why
R13	Semantic test matrix: call graphs, effect timing, capabilities, persistence, AOT, fuzz/ property tests	Parallel; each item must land its slice. This is how a green suite stops hiding a silent closure or an eager side effect.

Why this order

1. **R0 is the root.** A recursion marker, a host word, a book word, a type signature, and a word-table entry all need the same answer to "what does this name resolve to, and is this a call or a reference?" The group substrate is the one way recursion executes, so it lands before any surface or F# classification.
2. **R5 is the state foundation.** R2/R3/R4 add execution state; a root registry that lands after them is a root registry that already missed a root.
3. **R7 is evidence.** The performance plan is staged; the baseline tells us whether a change helped. Measuring after the redesign is not measurement.
4. **R9 is leverage.** The docs cost is paid by every session; splitting the roadmap and generating the status is independent of the runtime.
5. **R11 is last.** A rename is a review tax. Do it after the semantics and the tests have settled, with a codemod and a full suite.

First vertical slice

The first implementation slice is:

1. documentation: record the book non-recursion contract and the named refusal in `docs/books-and-spells.typ`;
2. tests: self, forward, and mutual calls are named refusals; an earlier-definition call still works;
3. implementation: the book loader/AST validation refuses a call to the current or a later definition;
4. specification: the books/spells status and counts.

D188 was the safety net while the durable callable/group model was built. D194 replaces it with a real recursive group; books use `logic`, `cycle`, and F#-style `and`.

Status: retired by D194. A book is one recursive group; self, forward, and mutual calls run. The D188 order-refusal is historical. The suite is 1530 checks.

Status: R0 first slice landed as D189. `Evaluator.executeWordStrict` is the lowered path's call; a `CallWord` that reaches a runtime lexical binding is `DynamicCall`; the book target is pushed and applied explicitly. The durable `Callable`/`Invoke` model remains the next R0 step.

Status: Slice 1 landed as D192. Image and package schema 2 carry `Unit`/`UnitGroup`, explicit arity and captures, `InitOrder`, and binding classification; ABI and image versions are 2.

Status: Slice 2 landed as D193. Group calls resolve through the unit's group, frame arguments and captures, and separate closure invocation from word calls. `UnitArityMismatch`, `UnitCaptureMissing`, and `EntryUnitArity` are named refusals.

Status: Slice 3 landed as D194. A book is one recursive group; self, forward, and mutual calls run. `logic` and `cycle` markers are checked against the book calls. D188's refusal is retired.

Status: Slice 4 landed as D195. Core `logic` and `cycle` definitions use the plain name for recursion; `logic` refuses a self-call, and `cycle ... and ...` stores a mutual group. There is no `recurse` word.

What this does not claim

This graph orders the complete plan; it does not reduce scope. The book recursion model, the F# abstraction and manifest, the effect-port ABI, and the verification directory move are all in scope. The graph only fixes the order so the shared substrate and the decisions are made once, at the root, and the leaves do not need to be rewritten.